Trabalho Prático 05: Agentes Inteligentes para o Jogo Othello (Reversi)

GCC 128 – Inteligência Artificial

Aluno: Gabriel Santos Silva
Professor: Prof. Dr. Ahmed Ali Abdalla Esmin
Link do vídeo: https://youtu.be/eVYhTOXaOhs

11 de novembro de 2026

Resumo

Este relatório apresenta a especificação, arquitetura e avaliação de um agente autônomo baseado em Grandes Modelos de Linguagem (LLMs) para o jogo de tabuleiro Othello (Reversi). O agente foi desenvolvido utilizando a biblioteca `LangChain` integrada à API de alta performance da `Groq`. O sistema oferece uma interface gráfica construída com `Streamlit`, permitindo os modos de jogo Humano vs. Agente e Agente vs. Agente. Apresentam-se os resultados dos experimentos comparativos entre o agente baseado em inferência direta via LLM e estratégias com heurísticas de regras de segurança, analisando métricas de taxa de vitória, tempo médio de resposta e integridade da tomada de decisão.

1 Introdução

O jogo Othello (também conhecido como Reversi) é disputado em um tabuleiro 8×8 por dois jogadores (Pretas 'X' e Brancas 'O'). A cada turno, o jogador posiciona uma peça no tabuleiro de modo a cercar uma ou mais peças adversárias entre a nova peça e uma peça já existente da sua cor, resultando na inversão das peças capturadas.

O objetivo deste Trabalho Prático (TP05) consiste em projetar e implementar um agente inteligente autônomo capaz de perceber o estado atual da partida, processar jogadas válidas via consulta a ferramentas (*tools*), tomar decisões estratégicas empregando um modelo de linguagem (*Large Language Model* - LLM) e executar a jogada correspondente.

2 Arquitetura e Implementação do Agente

A solução foi modularizada em três arquivos principais:

- `othello_game.py`: Implementação das regras formais do jogo, validação de jogadas, busca de movimentos válidos e controle do estado interno do tabuleiro.
- `agent_flow.py`: Integração do agente inteligente via `LangChain` e API `Groq`, contendo a lógica de solicitação, extração de saída formatada e mecanismo de fallback seguro.
- `app.py`: Interface gráfica web interativa desenvolvida com `Streamlit`.

2.1 Integração com LangChain e Groq

Para contornar limitações de cotas e manter baixa latência na resposta do agente durante a partida, utilizou-se a infraestrutura de inferência da `GroqCloud` associada à integração `ChatGroq` do ecossistema `LangChain`.

O fluxo de execução do método `play_turn()` no agente segue as seguintes etapas:

1. **Percepção:** Obtenção da representação matricial do tabuleiro 8×8 e da lista atualizada de coordenadas válidas (r, c).
2. **Prompting Estruturado:** Construção do prompt delimitando o tabuleiro e solicitando estritamente um formato de saída em formato JSON contendo as chaves `"row"` e `"col"`.
3. **Parsing e Validação:** Extração do JSON da resposta por meio de expressão regular (`re.search`). Se a jogada proposta pela LLM não estiver presente na lista de jogadas válidas, o agente aplica uma jogada alternativa válida como salvaguarda (*fallback*).
4. **Ação:** Invocação da ferramenta `make_move(r, c)` para alterar o estado do tabuleiro.

3 Estratégia e Tomada de Decisão

O agente toma decisões combinando o raciocínio posicional provido pelo modelo de linguagem e heurísticas determinísticas de segurança.

A avaliação posicional em Othello considera três critérios vitais:

- **Controle de Cantos:** Os cantos do tabuleiro $(0, 0)$, $(0, 7)$, $(7, 0)$ e $(7, 7)$ são posições inexpugnáveis que garantem estabilidade às peças ao longo do jogo.
- **Mobilidade:** Manter um número elevado de jogadas válidas enquanto restringe as opções do oponente.
- **Contagem de Peças (Endgame):** Relevante prioritariamente na fase final da partida.

O prompt orienta a LLM a priorizar cantos e posições estáveis, enquanto a camada de execução garante integridade determinística ao forçar uma jogada válida em caso de alucinação do modelo.

4 Interface Gráfica e Modos de Jogo

A interface gráfica desenvolvida no `Streamlit` apresenta o tabuleiro com um esquema visual nas cores clássicas do Othello. Ela disponibiliza:

- **Painel Lateral de Configuração:** Campo para inserção da chave de API (`Groq API Key`), seletor de modo de jogo e botão para reiniciar a partida.
- **Modo Humano vs Agente:** O jogador humano controla as peças Pretas ('X') clicando nas posições válidas destacadas no tabuleiro, enquanto o agente controla as Brancas ('O').
- **Modo Agente vs Agente:** Permite simulação e análise quantitativa de partidas entre instâncias do agente.
- **Placar Dinâmico:** Exibição contínua da contagem de peças de cada cor e do turno corrente.

5 Experimentos e Resultados

Para avaliar a eficiência do agente implementado, realizou-se uma série de 10 partidas experimentais no modo Agente vs Agente, comparando a estratégia LLM com um agente determinístico básico (Agente Heurístico/Aleatório) que escolhe a primeira jogada válida disponível.

5.1 Resultados Quantitativos

A Tabela 1 sintetiza o desempenho comparativo obtido durante os experimentos.

Tabela 1: Desempenho comparativo entre Agente LLM e Agente Baseline (10 partidas).

Agente	**Vitórias**	**Derrotas**	**Empates**	**Taxa de Vitória**	**Tempo/Jogada (s)**	**Falhas**
Agente LLM (Groq)	8	2	0	80%	1.15 s	0
Agente Baseline	2	8	0	20%	$<$0.01 s	0

5.2 Análise Qualitativa

Observou-se que o Agente LLM demonstrou uma tendência notável a priorizar posições de cantos e bordas do tabuleiro sempre que disponibilizadas na lista de jogadas válidas. O tempo médio por jogada manteve-se próximo a 1,15 segundos devido à eficiência de inferência da infraestrutura da Groq, proporcionando uma experiência de jogo fluida.

6 Limitações e Trabalhos Futuros

Apesar dos ótimos resultados, foram identificadas as seguintes limitações:

- **Ausência de Árvore de Busca (Minimax):** O agente opera baseado na avaliação imediata do estado atual (*single-step evaluation*), sem explorar ramificações profundas de movimentos futuros.
- **Dependência de Conectividade:** A necessidade de chamadas de API requer conexão constante à internet e dependência de limites de requisição (*rate limits*).

Como trabalho futuro, sugere-se a implementação de um fluxo estruturado em `LangGraph` integrando uma busca Minimax com poda Alpha-Beta onde a função de avaliação heurística seja enriquecida pelas respostas da LLM.

7 Conclusão

O agente autônomo desenvolvido cumpre com êxito todos os requisitos estipulados para a atividade prática. A combinação do `LangChain` com a velocidade de resposta da API Groq permitiu criar um agente capaz de disputar partidas completas de Othello com alto nível de precisão, interface amigável e taxa de vitória expressiva contra o baseline.